Unit-4
Flask Framework

Virtual Environment

A **virtual environment** in Python is an isolated environment on your computer, where you can run and test your Python projects. It allows you to manage project-specific dependencies without interfering with other projects or the original Python installation.

Think of a virtual environment as a separate container for each Python project. Each container:

- Has its own Python interpreter
- Has its own set of installed packages
- Is isolated from other virtual environments
- Can have different versions of the same package

Using virtual environments is important because:

- It prevents package version conflicts between projects
- Makes projects more portable and reproducible
- Keeps your system Python installation clean
- Allows testing with different Python versions

Creating Virtual Environment

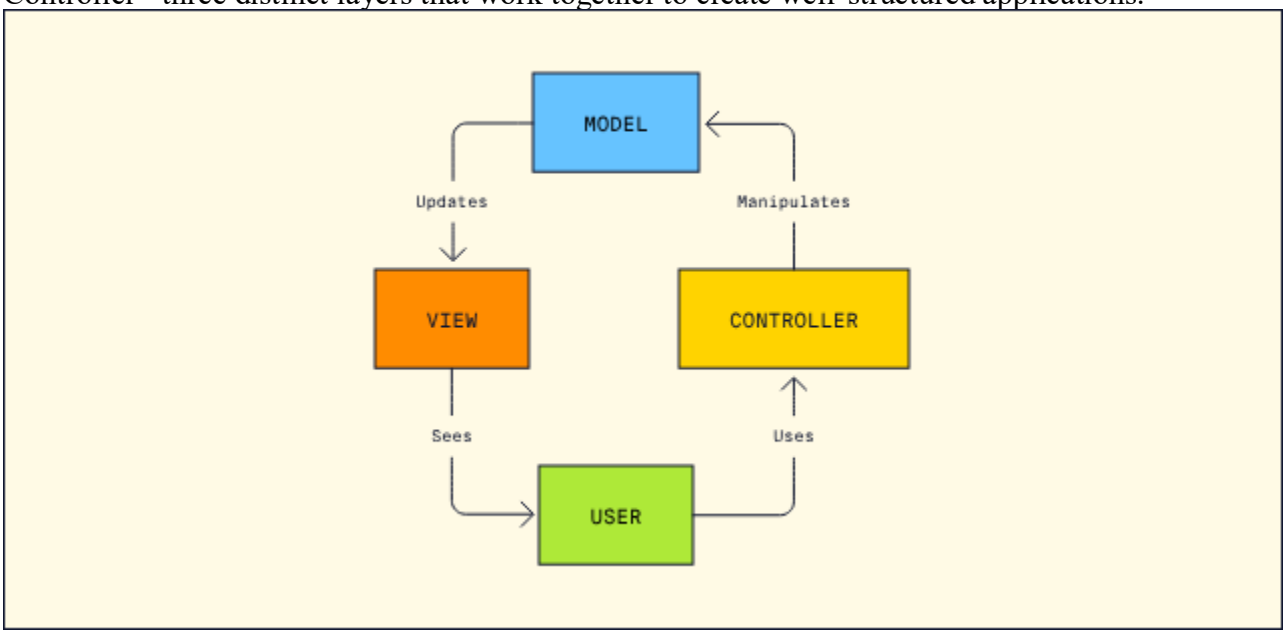
Throw the following commands:

- python -m venv myfirstproject
- myfirstproject\Scripts\activate

MVC Architecture

MVC architecture is a fundamental design pattern that helps developers organize code by separating an application into three interconnected components. MVC stands for Model, View,

Controller - three distinct layers that work together to create well-structured applications.



Components of MVC architecture

Model: data and business logic
The Model represents the data layer of your application. Model code typically reflects real-world things and contains the essential components that define what your application does.
Key responsibilities of the Model:
- Stores and manages application data
- Defines business rules and logic
- Handles data validation
- Manages database interactions
- Notifies other components when data changes

View: user interface and presentation
The View handles everything users see and interact with directly. View code is made up of all the functions that directly interact with the user, making your app look nice and defining how users see and interact with it.
Key responsibilities of the View:
- Displays data to users
- Handles user interface elements
- Manages layout and styling
- Responds to user interactions
- Updates when underlying data changes

Controller: application logic and coordination
The Controller acts as a liaison between the Model and the View, receiving user input and deciding what to do with it. It's the brains of the application that tie together the Model and the View.
Key responsibilities of the Controller:
- Processes user input
- Updates the Model based on user actions

- Selects appropriate Views to display
- Handles application flow and navigation
- Coordinates between Model and View components

Introduction to Flask

- **Flask** is a **lightweight and powerful** web framework for Python which is based on **MVC architecture**.
- It's often called a **"micro-framework"** because it provides the essentials for web development without unnecessary complexity.
- Unlike Django, which comes with built-in features like authentication and an admin panel, Flask keeps things minimal and lets us add only what we need.
- **Armin Ronacher**, who leads an international group of Python enthusiasts named **Pocco**, develops it.
- Flask is based on **Werkzeug WSGI toolkit** and **Jinja2 template engine**.
- Both are Pocco projects.

Installation of Flask in Windows

If pip is installed on the system, throw the following command to install Flask.

```
pip install flask
```

Creation of routing app

- Modern web frameworks use the routing technique to help a user remember application URLs.
- It is useful to access the desired page directly without having to navigate from the home page.
- The route() decorator in Flask is used to bind URL to a function.

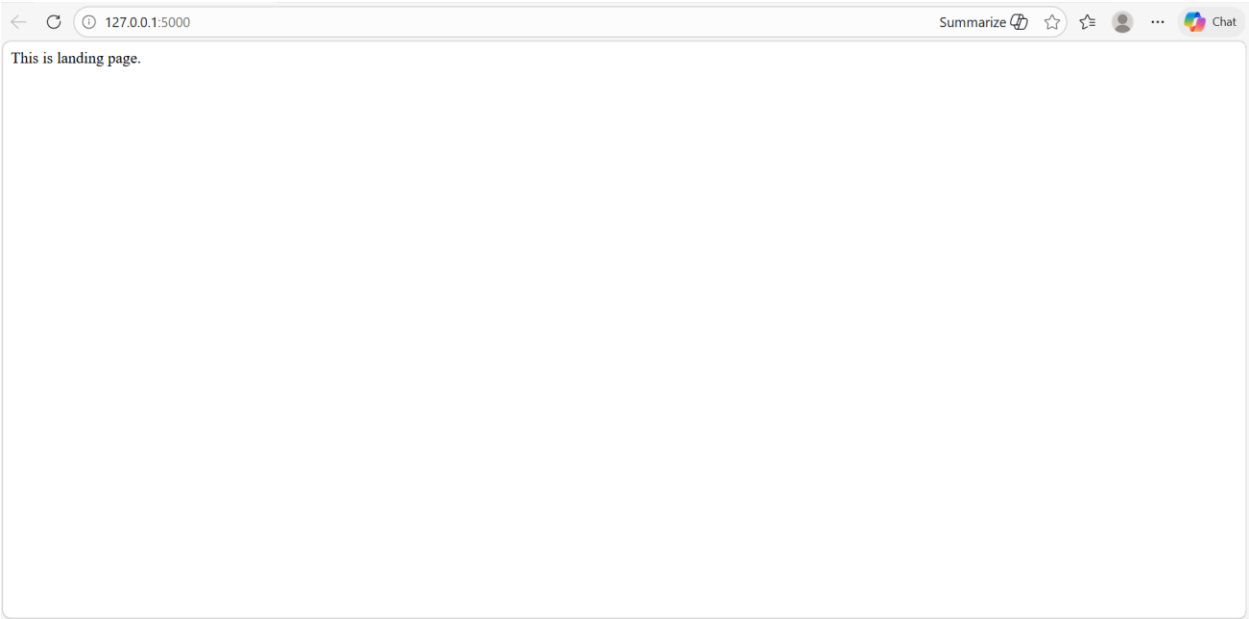
  Eg.
  ```
  @app.route(/hello)
  def hello_world():
    return hello world
  ```

URL Building

- The **url_for()** function is very useful for dynamically building a URL for a specific function.
- The function accepts the name of a function as first argument, and one or more keyword arguments, each corresponding to the variable part of URL.
- Usually in application, the URL ("/") is associated with the root URL.

  Eg.
  ```
  from flask import Flask, render_template

  app = Flask(__name__)

  @app.route('/')
  ```

```
def index():
    return "This is landing page."

@app.route('/about')
def about():
    return "This is about page."

if __name__ == '__main__':
    app.run(debug=True)
```



HTTP Methods

- HTTP methods are rules that define how clients and servers communicate in a client-server architecture.
- They allow browsers or apps to send requests and receive responses.
- For example, when you search on Google, your browser sends a request, and Google's server returns results.
- Http protocol is the foundation of data communication in world wide web.
- Different methods of data retrieval from specified URL are defined in this protocol.

Sr.No.	Methods & Description
1	**GET** Sends data in unencrypted form to the server. Most common method.
2	**HEAD** Same as GET, but without response body

3	**POST** Used to send HTML form data to server. Data received by POST method is not cached by server.
4	**PUT** Replaces all current representations of the target resource with the uploaded content.
5	**DELETE** Removes all current representations of the target resource given by a URL

Templates

- Generating HTML content from Python code is cumbersome, especially when variable data and Python language elements like conditionals or loops need to be put.
- This would require frequent escaping from HTML.
- This is where one can take advantage of Jinja2 template engine, on which Flask is based.
- One of Flask's powerful features is its ability to render HTML templates using Jinja2.
- Instead of returning plain strings in routes, we can use render_template() to serve HTML files dynamically.

  Eg.

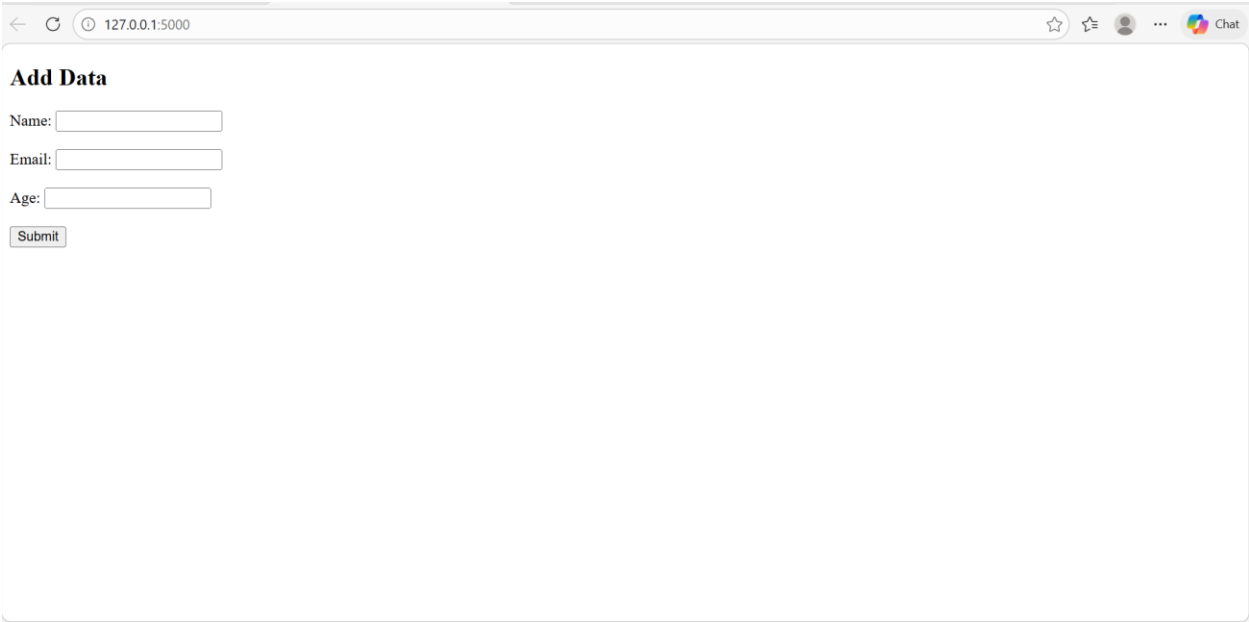
  app.py

```
from flask import Flask, render_template, request

app = Flask(__name__)

@app.route("/")
def index():
    return render_template("index.html")

if __name__ == "__main__":
    app.run(debug=True)
```

  index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Simple Form</title>
</head>
<body>
<h2>Add Data</h2>
<form method="POST">
    Name: <input type="text" name="name"><br><br>
    Email: <input type="email" name="email"><br><br>
    Age: <input type="number" name="age"><br><br>
```

```
    <button type="submit">Submit</button>
</form>
</body>
</html>
```



Working with static and media files

- In Flask, static files refer to files such as CSS, JavaScript, images, videos, and audio files that do not change dynamically.
- Flask provides a built-in way to serve these static files using the /static directory.
- Flask can also serve media files like images, videos, audio, text files, and PDFs using the static folder. Just like CSS and JavaScript, media files are stored in static/ and linked to HTML files.
- Steps to Serve Media Files:
    - Place all media files inside the static/ folder.
    - Link them in HTML using the /static/ path. Example: <img src="{{ url_for('static', filename='images/sample.jpg') }}" alt="Sample Image">
    - In main.py, create routes for any static files you need to render.
    - Run the Flask app, and media files will be served along with the web pages.

Eg.

```
@app.route("/image")
def serve_image():
    message = "Image Route"
    return render_template('image.html', message=message)
```

image.html

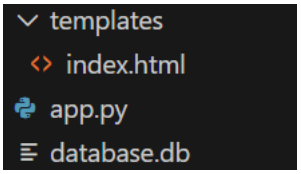
```
<html>
  <head>
    <title>Flask Static Demo</title>
    <link rel="stylesheet" href="/static/style.css" />
  </head>
  <body>
    <h1>{{message}}</h1>

    <img src="/static/cat.jpg" alt="Cat image" width="20%" height="auto" />

    <script src="/static/serve.js" charset="utf-8"></script>
  </body>
</html>
```

Flask App with Database connectivity and Sending Form Data to Template

- Python has an in-built support for SQlite.
- SQlite3 module is shipped with Python distribution.
- **What is ORM (Object Relation Mapping)?**
    - Most programming language platforms are object oriented.
    - Data in RDBMS servers on the other hand is stored as tables.
    - Object relation mapping is a technique of mapping object parameters to the underlying RDBMS table structure.
    - An ORM API provides methods to perform CRUD operations without having to write raw SQL statements.
- One can take advantage of Jinja2 template engine, on which Flask is based.
- It is one of Flask's powerful features to render HTML templates using Jinja2.
- The **jinja2** template engine uses the following delimiters for escaping from HTML.
    - {% ... %} for Statements
    - {{ ... }} for Expressions to print to the template output
    - {# ... #} for Comments not included in the template output
    - # ... ## for Line Statements

The URL rule to the **index()** function accepts the user input from the form. It is passed to the **index.html** template.

This is how the file structure of our app will look like after completion.



Eg.

index.html

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Simple Form</title>
</head>
<body>
<h2>Add Data</h2>
<form method="POST">
    Name: <input type="text" name="name"><br><br>
    Email: <input type="email" name="email"><br><br>
    Age: <input type="number" name="age"><br><br>
    <button type="submit">Submit</button>
</form>
<hr>
<h2>Display data</h2>
<table border="1">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Age</th>
    </tr>
    {% for user in users %}
    <tr>
        <td>{{ user[0] }}</td>
        <td>{{ user[1] }}</td>
        <td>{{ user[2] }}</td>
        <td>{{ user[3] }}</td>
    </tr>
    {% endfor %}
</table>
</body>
</html>
```

Create a new file named **app.py** and render HTML template to build the front end.

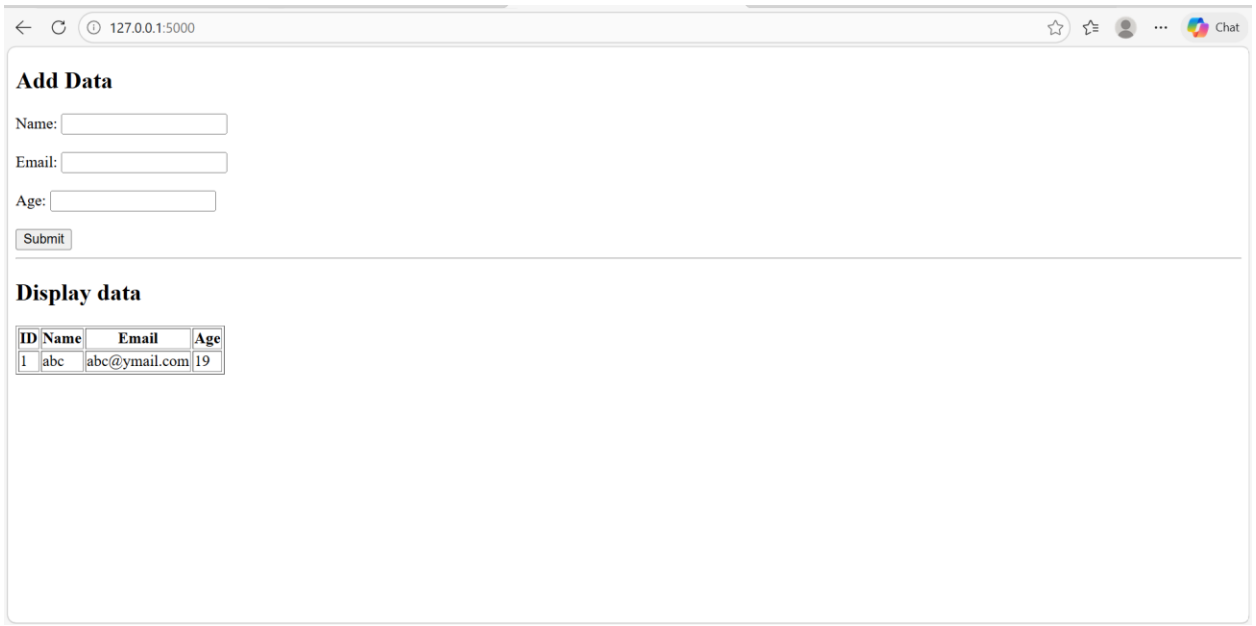
- To insert data, first create a database table with Name, Email and Age columns.
- Connect to the database using sqlite3.connect("database.db"). If the table doesn't exist, create it.
- Use a Jinja template loop in HTML to dynamically display database entries.
- The function selects all columns from the table and retrieves data using fetchall().

Eg.

app.py

```
from flask import Flask, render_template, request
import sqlite3

app = Flask(__name__)
```

```
db = sqlite3.connect("database.db", check_same_thread=False)
cursor = db.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY
AUTOINCREMENT, name TEXT, email TEXT, age INTEGER)")
db.commit()

@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":
        name = request.form["name"]
        email = request.form["email"]
        age = request.form["age"]

        cursor.execute(
            "INSERT INTO users (name, email, age) VALUES (?, ?, ?)",
            (name, email, age)
        )
        db.commit()

    data = cursor.execute("SELECT * FROM users").fetchall()

    return render_template("index.html", users=data)

if __name__ == "__main__":
    app.run(debug=True)
```



Handling Exceptions and Errors

If there is an error in the address or if there is no such URL then Flask has an **abort()** function used to exit with an error code.
Flask class has **abort()** function with an error code.
Flask.abort(code)
The **Code** parameter takes one of following values −

- **400** − for Bad Request
- **401** − for Unauthenticated
- **403** − for Forbidden
- **404** − for Not Found
- **406** − for Not Acceptable
- **415** − for Unsupported Media Type
- **429** − Too Many Requests

Eg.
app.py

```
from flask import Flask, redirect, url_for, render_template, request, abort
app = Flask(__name__)

@app.route('/')
def index():
   return render_template('log_in.html')

@app.route('/login',methods = ['POST', 'GET'])
def login():
   if request.method == 'POST':
      if request.form['username'] == 'admin' :
         return redirect(url_for('success'))
      else:
         abort(401)
   else:
      return redirect(url_for('index'))

@app.route('/success')
def success():
   return 'logged in successfully'

if __name__ == '__main__':
   app.run(debug = True)
```

Flash Message
A good GUI based application provides feedback to a user about the interaction. For example, the desktop applications use dialog or message box and JavaScript uses alerts for similar purpose.

Generating such informative messages is easy in Flask web application. Flashing system of Flask framework makes it possible to create a message in one view and render it in a view function called **next**.
A Flask module contains **flash()** method. It passes a message to the next request, which generally is a template.
flash(message, category)
Here,

- **message** parameter is the actual message to be flashed.
- **category** parameter is optional. It can be either error, info or warning.

In order to remove message from session, template calls **get_flashed_messages()**.
get_flashed_messages(with_categories, category_filter)
Both parameters are optional. The first parameter is a tuple if received messages are having category. The second parameter is useful to display only specific messages.
Eg.
login.html

```
<!doctype html>
<html>
  <body>
    <h1>Login</h1>

    {% if error %}
      <p><strong>Error:</strong> {{ error }}
    {% endif %}

    <form action = "" method = post>
      <dl>
        <dt>Username:</dt>
        <dd>
          <input type = text name = username
            value = "{{request.form.username }}">
        </dd>
        <dt>Password:</dt>
        <dd><input type = password name = password></dd>
      </dl>
      <p><input type = submit value = Login></p>
    </form>
  </body>
</html>
```

Working with Mails
Python, being a powerful language don't need any external library to import and offers a native library to send emails- "SMTP lib". "smtplib" creates a Simple Mail Transfer Protocol client session object which is used to send emails to any valid email id on the internet.
To install Flask-Mail, use the following command:

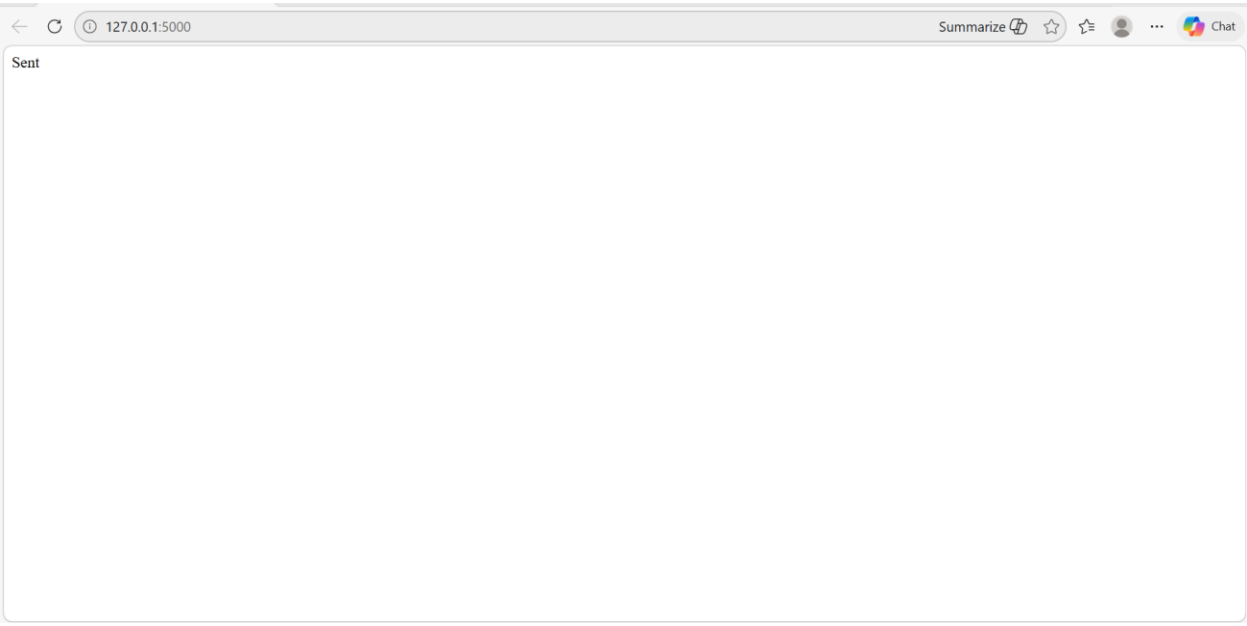
pip install Flask-Mail

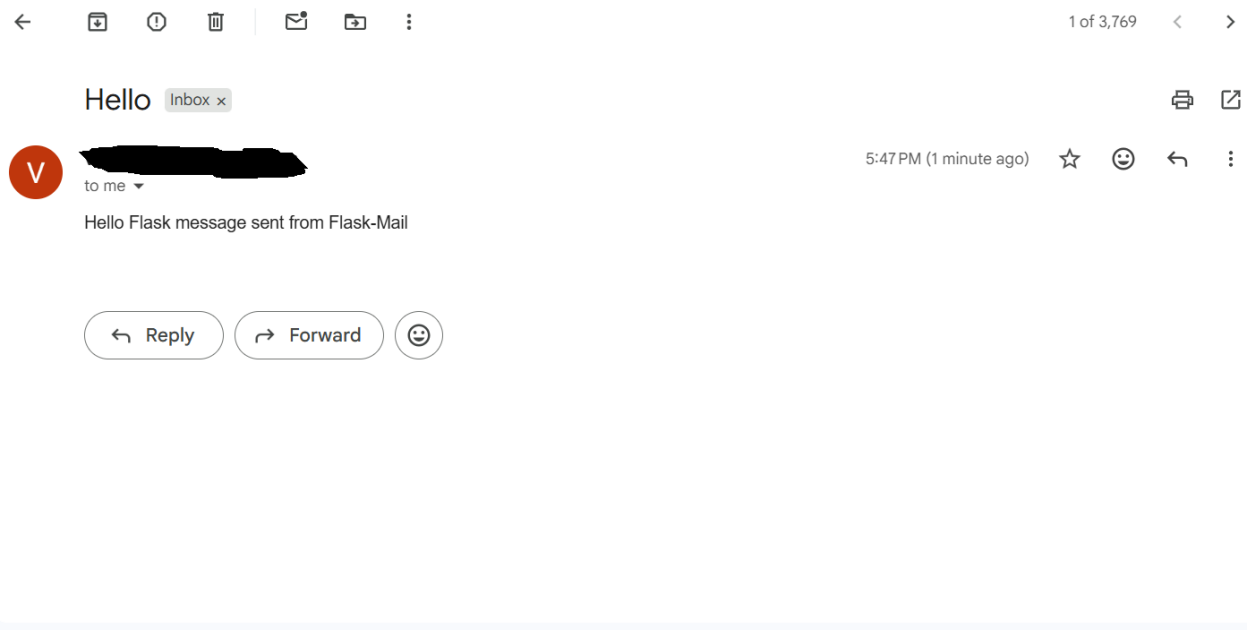
Configuring Flask-Mail
Flask-Mail is configured through the standard Flask config API. These are the available options (each is explained later in the documentation):
1) **MAIL_SERVER** : Name/IP address of the email server.
2) **MAIL_PORT** : Port number of server used.
3) **MAIL_USE_TLS** : Enable/disable Transport Security Layer encryption.
4) **MAIL_USE_SSL** : Enable/disable Secure Sockets Layer encryption
5) **MAIL_DEBUG** : Debug support. The default is Flask application's debug status.
6) **MAIL_USERNAME** : Username of the sender
7) **MAIL_PASSWORD** : The password of the corresponding Username of the sender.
8) **MAIL_ASCII_ATTACHMENTS** : If set to true, attached filenames converted to ASCII.
9) **MAIL_DEFAULT_SENDER** : sets default sender
10) **MAIL_SUPPRESS_SEND** : Sending suppressed if app.testing set to true
11) **MAIL_MAX_EMAILS** : Sets maximum mails to be sent

Classes in Flask-Mail: Mail Class : Manages email-messaging requirements Message Class: encapsulates an email message
Eg:
app.py

```
# importing libraries
from flask import Flask
from flask_mail import Mail, Message

app = Flask(__name__)
mail = Mail(app) # instantiate the mail class

# configuration of mail
app.config['MAIL_SERVER']='smtp.gmail.com'
app.config['MAIL_PORT'] = 465
app.config['MAIL_USERNAME'] = 'yourId@gmail.com'
app.config['MAIL_PASSWORD'] = '*****'
app.config['MAIL_USE_TLS'] = False
app.config['MAIL_USE_SSL'] = True
mail = Mail(app)

# message object mapped to a particular URL '/'
@app.route("/")
def index():
  msg = Message(
            'Hello',
            sender ='yourId@gmail.com',
            recipients = ['receiver'sid@gmail.com']
           )
```

```
    msg.body = 'Hello Flask message sent from Flask-Mail'
    mail.send(msg)
    return 'Sent'

if __name__ == '__main__':
    app.run(debug = True)
```





Authenticating and Authorizing users with Flask-login

We can implement **authentication**, **login/logout** functionality in flask app using **Flask-Login**.
install Flask, Flask-Login, Flask-SQLAlchemy and Werkzeug using this command:
pip install flask flask_sqlalchemy flask_login werkzeug

- **Flask-Login:** Manages user sessions and authentication.
- **Flask-SQLAlchemy:** Stores user data like usernames and passwords.
- **Werkeug:** Used for secure password hashing and verification.

Eg.
signup.html

```
<!doctype html>
<html lang="en">

<head>
   <title>Signup</title>
</head>

<body>
   <h1>Signup</h1>
   <form method="POST">
   <label>Email address</label>
   <input type="email" name="email"><br>
   <label>Password</label>
   <input type="password" name="password"><br>
   <label>Confirm Password</label>
   <input type="password" name="confirmPassword"><br><br>
   <button type="submit">Signup</button> <br><br>
   <div id="signup" class="form-text">Already have the account! Login.</div>
   <button type="button" onclick="window.location.href='{{url_for('login')}}';">Login</button>
   </form>
   {% if errorMessage %} <p style="color: red;">{{ errorMessage }}</p>
   {% endif %}
</body>
</html>
```

login.html

```
<!doctype html>
<html lang="en">

<head>
   <title>Login</title>
</head>

<body>
```

```
    <h1>Login</h1>
    <form action="{{ url_for('login') }}" method="POST">
        <label>Email address</label>
        <input type="email" name="email"><br>
        <label>Password</label>
        <input type="password" name="password"><br>
        <button type="submit">Login</button> <br><br>
        <div id="signup">Don't have the account! Signup.</div>
        <button type="button"
onclick="window.location.href='{{url_for('signup')}}';">Signup</button>
    </form>
    {% if errorMessage %}
        <p style="color: red;">{{ errorMessage }}</p>
    {% endif %}
</body>
</html>
```

index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Simple Form</title>
</head>
<body>
<h2>Add Data</h2>
<form method="POST">
    Name: <input type="text" name="name"><br><br>
    Email: <input type="email" name="email"><br><br>
    Age: <input type="number" name="age"><br><br>
    <button type="submit">Submit</button>
</form>
<hr>
<h2>Display data</h2>
<table border="1">
    <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Age</th>
    </tr>
    {% for user in students %}
    <tr>
        <td>{{ user[0] }}</td>
        <td>{{ user[1] }}</td>
        <td>{{ user[2] }}</td>
```

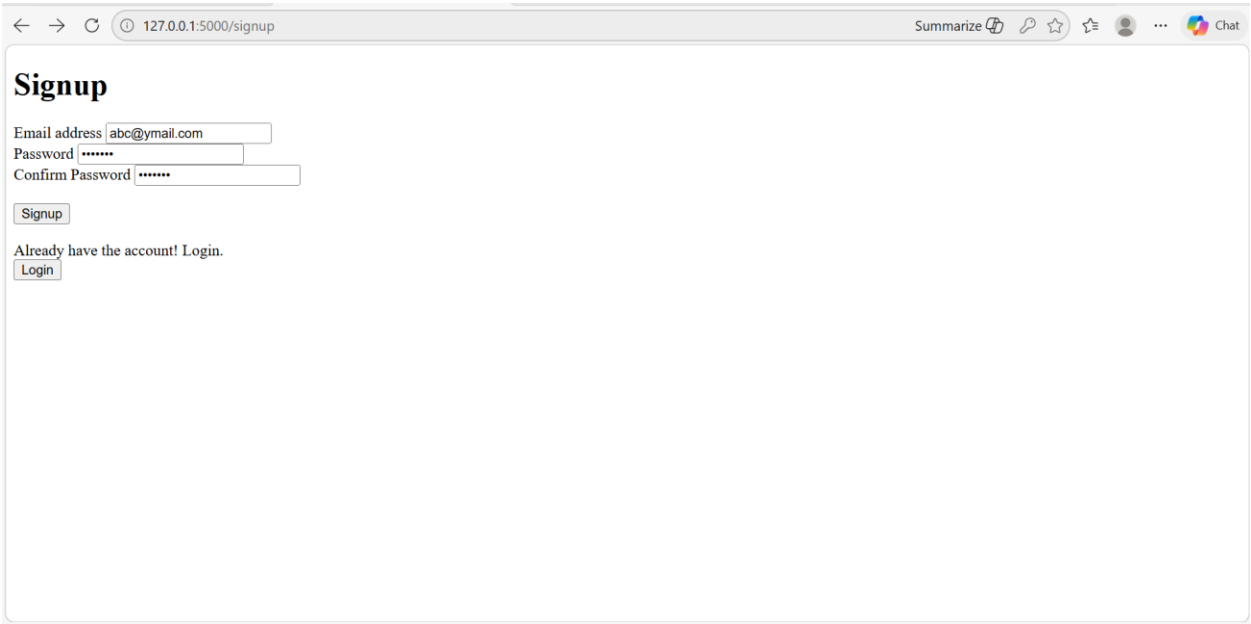
```
        <td>{{ user[3] }}</td>
    </tr>
    {% endfor %}
</table>
<br><br>
<a href="{{ url_for('logout') }}">
    <button>Logout</button>
</a>
</body>
</html>
```

app.py

```
from flask import Flask, render_template, request, redirect, url_for
import sqlite3
from werkzeug.security import generate_password_hash, check_password_hash
from flask_login import LoginManager, login_user, logout_user, current_user, login_required,
UserMixin

app = Flask(__name__)
app.secret_key = 'secretKey'

db = sqlite3.connect("database.db", check_same_thread=False)
db.row_factory = sqlite3.Row
cursor = db.cursor()
cursor.execute("CREATE TABLE IF NOT EXISTS STUDENTS (id INTEGER PRIMARY KEY
AUTOINCREMENT, name TEXT, email TEXT, age INTEGER)")
cursor.execute('CREATE TABLE IF NOT EXISTS USERS (email TEXT PRIMARY KEY,
password TEXT NOT NULL)')

db.commit()

login_manager = LoginManager()
login_manager.init_app(app)
login_manager.login_view = 'login'

class User(UserMixin):
    def __init__(self, email):
        self.id = email

@login_manager.user_loader
def load_user(user_id):
    user = cursor.execute('SELECT * FROM USERS WHERE email = ?', (user_id,)).fetchone()
    if user:
        return User(user['email'])
    else:
```

```
        None

@app.route('/')
def root():
    if current_user.is_authenticated:
        return redirect(url_for('index'))
    else:
        return render_template("login.html")

@app.route('/signup', methods = ['GET','POST'])
def signup():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        confirmPassword = request.form['confirmPassword']

        if password != confirmPassword:
            return render_template('signup.html', errorMessage = 'Passwords do not match.')

        hashed_password = generate_password_hash(password)
        existing_user = cursor.execute('SELECT 1 FROM USERS WHERE email = ?',
(email,)).fetchone()

        if existing_user:
            return render_template('signup.html', errorMessage = 'User already exists.')
        else:
            cursor.execute('INSERT INTO USERS (email, password) VALUES (?,?)', (email,
hashed_password))
            db.commit()
            return redirect(url_for('login'))
    return render_template('signup.html')

@app.route('/login', methods = ['GET', 'POST'])
def login():
    if request.method == 'POST':
        email = request.form['email']
        password = request.form['password']
        user = cursor.execute('SELECT * FROM USERS WHERE email = ?', (email,)).fetchone()
        if user and check_password_hash(user['password'], password):
            login_user(User(user['email']))
            return redirect(url_for('index'))
        else:
            return render_template('login.html', errorMessage = 'Invalid credentials.')
    return render_template('login.html')

@app.route("/index", methods=["GET", "POST"])
```

```
@login_required
def index():
    if request.method == "POST":
        name = request.form["name"]
        email = request.form["email"]
        age = request.form["age"]

        cursor.execute(
            "INSERT INTO STUDENTS (name, email, age) VALUES (?, ?, ?)",
            (name, email, age)
        )
        db.commit()

    data = cursor.execute("SELECT * FROM STUDENTS").fetchall()

    return render_template("index.html", students=data)

@app.route('/logout')
@login_required
def logout():
    logout_user()
    return redirect(url_for('login'))

if __name__ == "__main__":
    app.run(debug=True)
```



127.0.0.1:5000/login

Login

Email address abc@ymail.com
Password •••••••
Login

Don't have the account! Signup.
Signup

127.0.0.1:5000/index

Add Data

Name:

Email:

Age:

Submit

Display data

ID	Name	Email	Age
1	abc	abc@ymail.com	19

Logout